

Activity

Firhat Hidayat MT



INSTITUT
TEKNOLOGI
HARAPAN
BANGSA

Veritas vos liberabit

Activity Teori

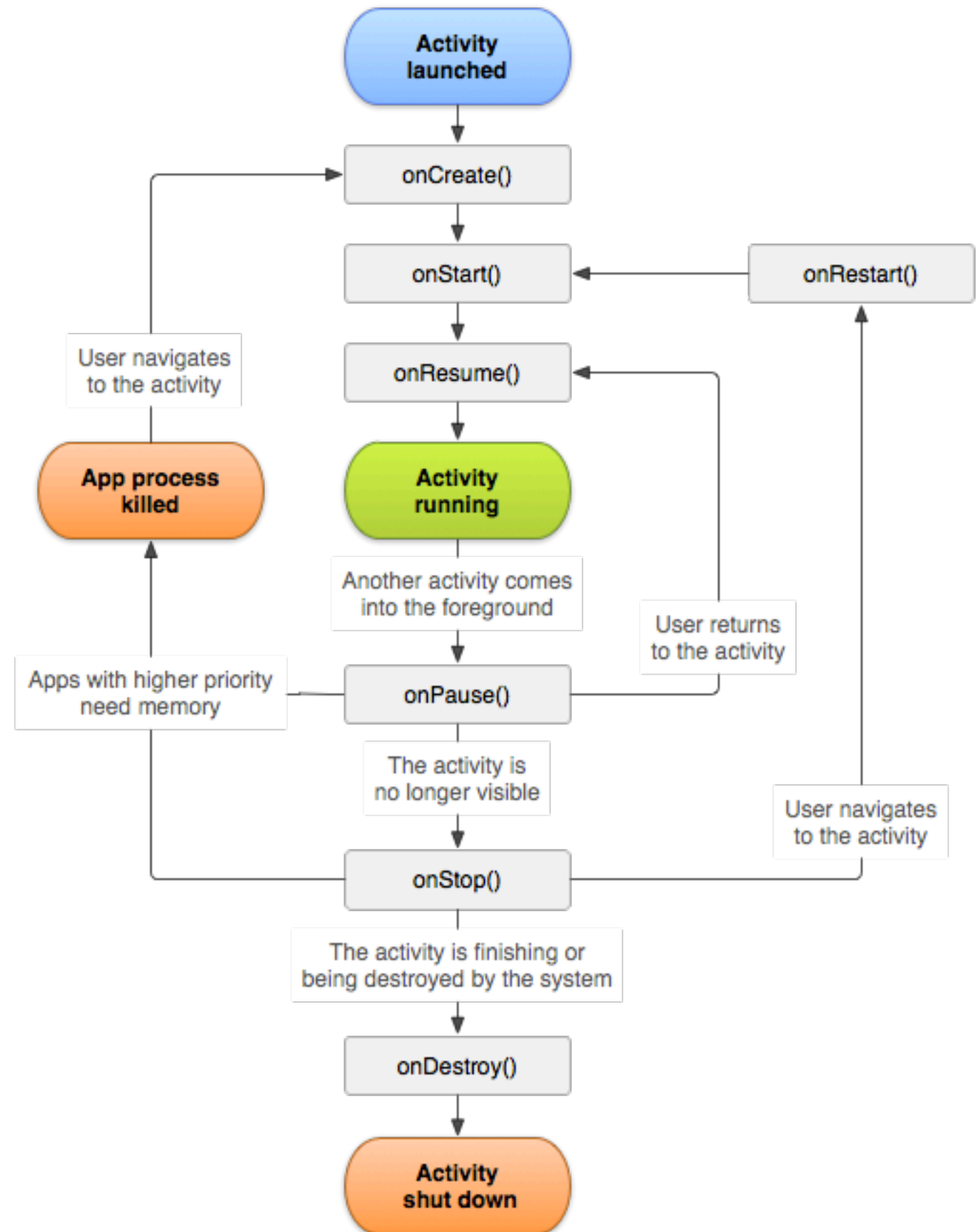
- komponen di Android yang berfungsi untuk menampilkan user *interface* ke layar handset Android pengguna. contoh : saat Anda melihat daftar percakapan pada aplikasi *chat* atau daftar *email* pada aplikasi Gmail di ponsel Android Anda.
- Umumnya dalam sebuah aplikasi terdapat lebih dari satu activity yang saling terhubung dengan tugas yang berbeda-beda.
- merupakan salah satu komponen penting Android yang memiliki daur hidup (*life cycle*) dalam sebuah *stack* pada *virtual sandbox* yang disiapkan oleh *Dalvik Virtual Machine* (DVM) atau *Android Runtime* (ART)
- bersifat *last in first out*.

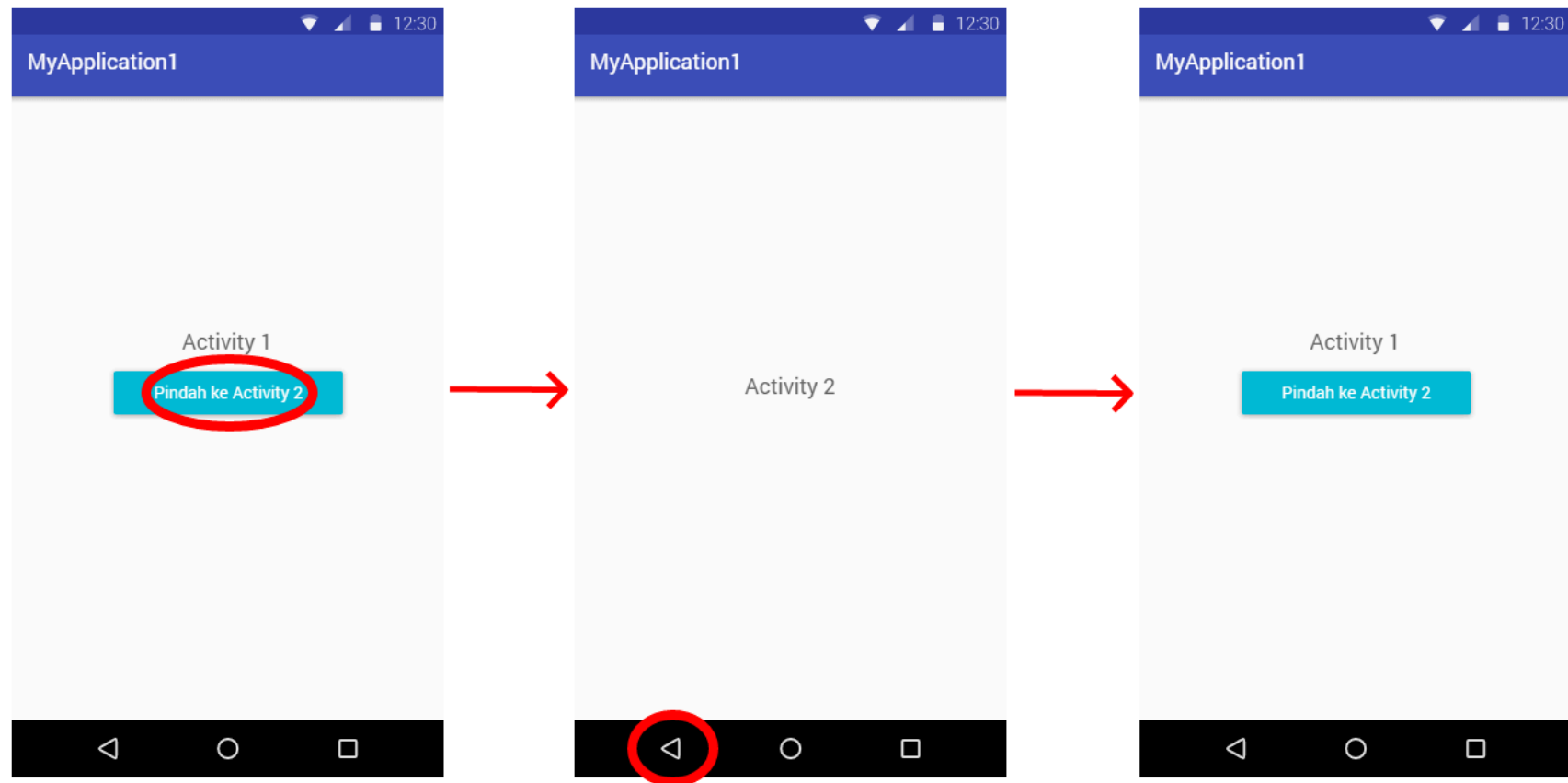
Activity Teori (cont)

- activity selalu memiliki satu layout *user interface* dalam bentuk ***berkas xml***.
- Suatu aplikasi Android bisa memiliki lebih dari satu activity dan harus terdaftar di berkas ***AndroidManifest.xml*** sebagai sub aplikasi.
- Sebuah kelas Java dinyatakan sebuah activity jika mewarisi (*extends*) superclass Activity atau turunannya seperti AppCompatActivity dan FragmentActivity.

Activity Lifecycle

- Developer yang baik harus mengetahui secara detail tentang *life cycle* sebuah activity.
- Terutama untuk melakukan aksi yang tepat, saat terjadi perubahan *state* activity. *Callback methods* yang ada dapat digunakan untuk melakukan beragam proses terkait *state* dari activity.
- Misalnya melakukan semua inisialisasi komponen di `onCreate()`, melakukan *disconnect* terhadap koneksi ke *server* pada `onStop()` atau `onDestroy()` dan lain sebagainya.
- Pemahaman yang baik tentang daur hidup activity akan membuat implementasi rancangan aplikasi Anda menjadi lebih baik. Hal ini juga akan meminimalisir terjadinya *error/bug/force close* yang tidak diinginkan.





<i>Gambar 1</i>	<i>Gambar 2</i>	<i>Gambar 3</i>
Aktif: Activity 1 <i>onCreate()</i> → <i>onStart()</i> → <i>onResume()</i>	Aktif: Activity 2 Stack append: Activity 2 [<i>onResume()</i>]	Activity 1 <i>onStop()</i> → <i>onRestart()</i> → <i>onStart()</i> → <i>onResume()</i>
Aksi: <i>Klik Button1</i> (Pindah)	Aksi: <i>Klik Hardware Back Button</i>	Aktif: Activity 1
Stack append: Activity 1 [<i>onStop()</i>]	Activity 2 [<i>finish()</i>] Stack pop: Activity 2 [<i>onDestroy()</i>]	

Last In, First Out (LIFO)

- **Gambar 1**

Jika Anda memiliki sebuah aplikasi yang terdiri dari 2 activity, maka activity pertama akan dijalankan setelah pengguna meluncurkan aplikasi melalui ikon aplikasi di layar *device*. Activity yang ada saat ini berada pada posisi activity *running* setelah melalui beberapa *state* onCreate (*created*) → onStart (*started*) → onResume (*resumed*) dan masuk ke dalam sebuah *stack* activity.

Bila pada activity pertama Anda menekan sebuah tombol untuk menjalankan activity kedua, maka posisi *state* dari activity pertama berada pada posisi *stop*. Saat itu, *callback* onStop() pada activity pertama akan dipanggil.

Ini terjadi karena activity pertama sudah tidak berada pada layar *foreground* / tidak lagi ditampilkan. Semua informasi terakhir pada activity pertama akan disimpan secara otomatis.

Sementara itu, activity kedua masuk ke dalam *stack* dan menjadi activity terakhir yang masuk.

- **Gambar 2**

Activity kedua sudah muncul di layar sekarang. Ketika Anda menekan tombol *back* pada *physical button* menu utama atau menjalankan metode finish(), maka activity kedua Anda akan dikeluarkan dari *stack*.

Pada kondisi di atas, *state* activity kedua akan berada pada *destroy*. Oleh karenanya, metode onDestroy() akan dipanggil.

Kejadian keluar dan masuk *stack* pada proses di atas menandakan sebuah model *Last In, First Out*. Activity kedua menjadi yang terakhir masuk *stack* (*Last In*) dan yang paling pertama keluar dari *stack* (*First Out*).

- **Gambar 3**

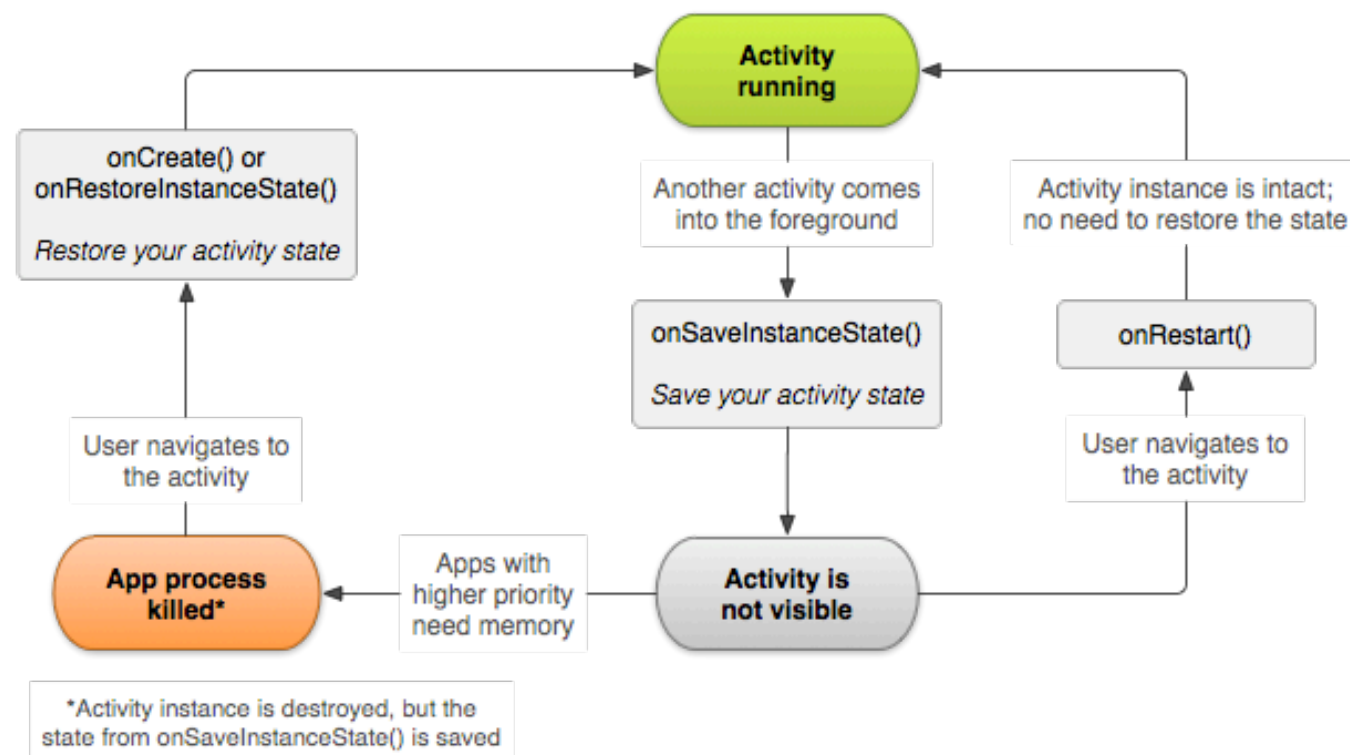
Activity Pertama akan dimunculkan kembali di layar setelah melalui beberapa *state* dengan rangkaian *callback method* yang terpanggil, onStop → onRestart → onStart → onResume.

Saving Activity State

- Ketika sebuah activity mengalami *pause* kemudian *resume*, maka state dari sebuah activity tersebut dapat terjaga. Sebabnya, obyek activity masih tersimpan di *memory* sehingga dapat dikembalikan *state*-nya.
- Dengan menjaga *state* dari activity, maka ketika activity tersebut ditampilkan, kondisinya akan tetap sama dengan kondisi sebelumnya.
- Akan tetapi ketika sistem menghancurkan activity untuk keperluan memori misalnya karena memori habis, maka obyek activity dihancurkan. Alhasil, ketika activity ingin ditampilkan kembali diperlukan proses *re-create* activity yang dihancurkan tadi.
- Kejadian di atas adalah hal yang lumrah terjadi. Oleh karena itu, perubahan yang terjadi pada activity perlu disimpan terlebih dahulu sebelum ia dihancurkan. Di sinilah metode `onSaveInstanceState()` digunakan.
- Dalam `onSaveInstanceState` terdapat `bundle` yang dapat digunakan untuk menyimpan informasi. Informasi dapat disimpan dengan memanfaatkan fungsi seperti `putString()` dan `putInt()`.
- Ketika activity di-*restart*, `bundle` akan diberikan kepada metode `onCreate` dan `onRestoreInstanceState`. `Bundle` tersebut akan dimanfaatkan untuk mengembalikan kembali perubahan yang telah terjadi sebelumnya.

Saving Activity State (cont)

- Proses penghancuran activity dapat juga terjadi ketika terdapat perubahan konfigurasi seperti perubahan orientasi layar (*portrait-landscape*), *keyboard availability*, dan perubahan bahasa. Penghancuran ini akan menjalankan *callback method* **onDestroy** dan kemudian menjalankan onCreate. Penghancuran ini dimaksudkan agar activity dapat menyesuaikan diri dengan konfigurasi baru yang muncul pada kejadian-kejadian sebelumnya.
- Hal yang perlu diingat ketika menggunakan onSaveInstanceState adalah untuk tidak menyimpan data yang besar pada bundle. Contohnya, hindari penyimpanan data *bitmap* pada bundle. Bila data pada bundle berukuran besar, proses serialisasi dan deserialisasi akan memakan banyak memori.



More resource

- <https://developer.android.com/guide/components/activities.html>
- <https://developer.android.com/guide/components/activities.html#Lifecycle>